

Supplementary material: complete proofs

The State Is Not the History, the Source Is: Identity, Responsibility, and Minimal Revision in Processes

Jian Gao (高健)

This supplement gives complete written proofs of the results of §6.5, the end of §7.3, §8.5 and §9 of the main text. The main text states the results and gives proof ideas; here each result is given its objects and domain, actual inputs and quantifiers, key lemmas, decisive derivation, and the direct use of the same object in later results, and is finally matched to its formal proof. Notation is that of the main text, and numbers of definitions, theorems and equations refer to the main text. The proofs of §2–§5, §6.1–§6.4, §7.1–§7.3 and §8.1–§8.4 are complete in the main text and Appendices A–C and are not repeated.

The “formal correspondence” at the end of each section lists the Lean declarations that carry the result; the source commit and full path are in Appendix D.2 of the main text, and the public reproduction entry is in its “Code and data availability”.

S0 Common tools

S0.1 Remaining count and execution. Proposition 3.7 gives: every execution step decreases the remaining count by exactly 1; constants have no execution step; the left-to-right execution $\text{exec}_\rho(e) : e \rightarrow \text{const}(\text{ev}_\rho e)$ has length $r(e)$; every execution trace is a derivation, so its endpoints evaluate equally (Proposition 3.6(1), below called **soundness**).

S0.2 The pair language preserves the remaining count. The pairing $e \mapsto \tilde{e}$ of Appendix A.2 replaces variables, constants, addition, linear and bilinear nodes one by one by nodes of the same kind on pair values, so $r(\tilde{e}) = r(e)$ (structural induction on e ; every node contributes the same count), and

$$\text{ev}_{(\rho, \varepsilon)}(\tilde{e}) = (\text{ev}_\rho(e), \text{eff}_{\rho, \varepsilon}(e)). \quad (\text{S0.1})$$

S0.3 Relation image of a trace and the updated reading. Let $t : e \rightarrow e'$ be an execution trace in environment ρ . Every execution step is a derivation certificate whose relation image is the difference of its endpoints; the relation image of the whole trace telescopes to $e - e'$. By soundness, $e - e' \in \ker(\text{ev}_\rho)$. Given an increment ε , the update equation of Proposition 3.3 gives

$$\text{ev}_{\rho+\varepsilon}(e - e') = \text{ev}_\rho(e - e') + \text{eff}_{\rho, \varepsilon}(e - e') = \text{eff}_{\rho, \varepsilon}(e) - \text{eff}_{\rho, \varepsilon}(e'). \quad (\text{S0.2})$$

In particular, when $e' = \text{const}(c)$ we have $\text{eff}(e') = 0$, and the updated value is $\text{eff}_{\rho, \varepsilon}(e)$.

S0.4 Coimages and induced maps. For a $\mathbb{Z}$ -linear map $f : X \rightarrow Y$, write $\text{Res}(f) := X / \ker f$ for its coimage and $\text{can}_f : X \rightarrow \text{Res}(f)$ for the quotient map; $\text{Res}(f) \cong \text{range}(f)$. If $f' \circ s = t \circ f$ (call (s, t) a morphism from f to f'), then $s(\ker f) \subseteq \ker f'$, so s induces $\bar{s} : \text{Res}(f) \rightarrow \text{Res}(f')$ with $\bar{s} \circ \text{can}_f = \text{can}_{f'} \circ s$. Moreover $\text{can}_f(x) = 0$ iff $f(x) = 0$.

S0.5 Lifting residual. Let (s, t) be a morphism from $f : C \rightarrow D$ to $f' : C' \rightarrow D'$. For $c \in C$ its **fibre** is $\text{Fib}_f(c) := \{x : f(x) = f(c)\}$. The map s sends $\text{Fib}_f(c)$ into $\text{Fib}_{f'}(sc)$ and $\ker f$ into $\ker f'$. For $y \in \text{Fib}_{f'}(sc)$ put

$$\text{lift}(c, y) := [y - s(c)] \in \ker f' / s(\ker f).$$

Lemma S0.5. $\text{lift}(c, y) = 0$ iff there is $x \in \text{Fib}_f(c)$ with $s(x) = y$.

Proof. If $y - s(c) = s(k)$ with $k \in \ker f$, take $x = c + k$; then $f(x) = f(c)$ and $s(x) = y$.

Conversely, if $x \in \text{Fib}_f(c)$ and $s(x) = y$, then $k = x - c \in \ker f$ and $y - s(c) = s(k)$. ■

S0.6 Action-word model. Let X be an additive group with a family $(A_\ell)_{\ell \in L}$ of additive actions and an additive read-out $\text{rd} : X \rightarrow Y$. For a word $w = \ell_1 \cdots \ell_n$ (acting first by ℓ_1) write $A_w := A_{\ell_n} \cdots A_{\ell_1}$. Let $K := \{x : \forall w, \text{rd}(A_w x) = 0\}$.

Lemma S0.6. (i) K is closed under every A_ℓ , so every A_ℓ descends to $\text{Mod} := X/K$; (ii) $[x] = [y]$ iff $\forall w, \text{rd}(A_w x) = \text{rd}(A_w y)$; (iii) the map sending $[x]$ to the family of read-outs $(w \mapsto \text{rd}(A_w x))$ is injective, so Mod is linearly isomorphic to the image of this family (the completion carrier of all word read-outs), and the isomorphism commutes with every action.

Proof. (i) If $x \in K$ then $\text{rd}(A_w A_\ell x) = \text{rd}(A_{\ell w} x) = 0$ for every w . (ii) By the definition of K and linearity. (iii) By (ii); commutation is checked word by word on representatives. ■

S0.7 Effect-history equation. Let also (U_ℓ) be additive actions on W and $M : X \rightarrow W$ an additive observation. For a point x and a word w let the total residual be $R(w, x) := U_w M(x) - M(A_w x)$ and the single-letter residual $r(\ell, x) := U_\ell M(x) - M(A_\ell x)$; define recursively

$$\text{Acc}([], x) = 0, \quad \text{Acc}(\ell w, x) = U_w r(\ell, x) + \text{Acc}(w, A_\ell x).$$

Lemma S0.7. $\text{Acc}(w, x) = R(w, x)$; the history recording $(\ell_k, x_{k-1}, r(\ell_k, x_{k-1}))$ has length $|w|$.

Proof. Induction on the length of w :

$$\begin{aligned} R(\ell w, x) &= U_w U_\ell M(x) - M(A_w A_\ell x) \\ &= U_w (U_\ell M(x) - M(A_\ell x)) + (U_w M(A_\ell x) - M(A_w A_\ell x)); \end{aligned}$$

the first term is $U_w r(\ell, x)$ and the second is $R(w, A_\ell x)$. ■

S0.8 Joint action-word carrier. Let I, H, Q be additive groups with three families of actions $(\alpha_\ell), (\eta_\ell), (\mu_\ell)$ and two additive read-outs $c : I \rightarrow H, m : I \rightarrow Q$. On $I \times H \times Q$ put $\theta_\ell := \alpha_\ell \times \eta_\ell \times \mu_\ell$ and $\text{seed}(i) := (i, c(i), m(i))$. The **complete orbit carrier** $\mathcal{C}$ is the submodule spanned by all $\theta_w(\text{seed } i)$.

Lemma S0.8. (i) $\mathcal{C}$ is the smallest submodule containing all seeds and closed under every θ_ℓ ; (ii) the three projections π_I, π_H, π_Q commute with the actions of words; (iii) the incidence element $\text{inc}(w, i) := \theta_w(\text{seed } i) - \text{seed}(\alpha_w i)$ has I -component zero, and its H - and Q -components are $\eta_w c(i) - c(\alpha_w i)$ and $\mu_w m(i) - m(\alpha_w i)$.

Proof. (i) The generated submodule contains the seeds by definition; any submodule containing the seeds and closed under the actions contains every $\theta_w(\text{seed } i)$ and hence their span. $\mathcal{C}$ itself is closed because the image of a generator is again a generator. (ii) θ_ℓ is a product. (iii) Componentwise from (ii). ■

S1 Theorem 6.10: birth and payment of a responsibility without a prior bearer

Objects and domain. Fix a world network N , an authoritative root on it with complete whole-ledger registration, an origin current c , and a reader that gives a complete environment-expression pair at every occurrence of the root at c . Let o be the canonical occurrence at c , (ρ, e) the output of the reader at o , $B = r(e)$, and $\Lambda = \Lambda_{\rho, e}$ the execution debt law of Definition 6.9.

Inputs and quantifiers. The theorem holds for all such data. No old responsibility row, registered inquiry, program table, table of future requests or empty-coverage certificate is supplied; the origin support stays fixed during the whole calculation.

Construction.

1. *Extended world and vocabulary.* The extended world $N[\Lambda]$ is given by Definition 6.3: its supports are $\text{Sup} \times \text{Option}(\text{debt states})$. The currents of the vocabulary are the debt states of Λ , i.e., pairs (e', t) with $t : e \rightarrow e'$ a paid execution trace; the initial current is (e, id) . The support of the current x is $(\text{supp}(o), \text{some } x)$.
2. *Event algebra.* At the current $x = (e', t)$ the **source-selected action** is the output of the executor: a settlement if e' is a constant, and otherwise the first step $s : e' \rightarrow e''$ of the execution of Proposition 3.7. The successor current x^+ is x at a settlement and $(e'', t \cdot s)$ at an execution step. Compilation sends a settlement to the identity transport of Λ (fourth item of Definition 6.9) and an execution step to a native write.
3. *Whole-ledger evolution.* $\text{whole}(x)$ sends every row on the support $(\text{supp}(o), x)$ to $(\text{supp}(o), x^+)$: the mathematical row $\text{row}(x)$ (the debt row of Λ at state x) goes to $\text{row}(x^+)$, and every old row is transported unchanged. Destination and origin are mutually inverse.
4. *Patch, remainder and certificate.* The finite patch contains exactly one generated row, the mathematical row; all other rows are covered by the same-source transported remainder, whose occurrence is the evidence “this row belongs to the old ledger”—no finiteness of the old ledger is needed. The patch folds back to the whole-ledger evolution: $\text{fold}(\text{patch}(x)) = \text{whole}(x)$. The restructuring certificate is generated by injectivity of the destination map of the whole-ledger evolution.
5. *Compiler, root and process.* The compiler emits patch and whole ledger at every current; the ledger root consists of source and compiler; the authoritative root and the living process are

obtained by the standard construction of root registration. The state of the process at count k is written x_k , with $x_0 = (e, \text{id})$ and $x_{k+1} = x_k^+$.

Key lemmas.

Lemma S1.1 (one-step budget). $r(x^+) = r(x) - 1$ (natural subtraction); if $r(x) = 0$ then $x^+ = x$.

Proof. If $r(x) > 0$ then e' is not a constant, the source-selected action is an execution step, and by S0.1 the count drops by one; if $r(x) = 0$ then e' is a constant, the action is a settlement and the successor is unchanged. ■

Lemma S1.2 (budget and history). For every k : $r(x_k) = B - k$ and $|t_k| + r(x_k) = B$, where t_k is the paid trace of x_k .

Proof. Induction on k . For $k = 0$, t_0 is empty and $r(x_0) = B$. Induction step: if $r(x_k) > 0$, the execution step lengthens the trace by one and decreases the count by one; if $r(x_k) = 0$, the state is unchanged and $B - (k + 1) = 0$. ■

Decisive derivation.

1. *Birth.* By construction steps 2–4, every current emits exactly one event; compilation is the identity transport at a settlement and a native write at an execution step. The patch contains exactly one generated row; all old rows are kept as the remainder, and mutual inversion of origin and destination gives the restructuring certificate. The original support is unchanged (the first component of the support is always $\text{supp}(o)$), and the law surface, old projections and type and value observations at the origin are kept as root readings.
2. *Payment.* The same-debt current at visit k is $\text{row}(x_k)$; the destination equation $\text{whole}(x_k)(\text{row}(x_k)) = \text{row}(x_{k+1})$ gives by induction on k its lineage to $\text{row}(x_0)$. Its budget is $r(x_k) = B - k$ (Lemma S1.2). For $k < B$, $r(x_k) > 0$, the source-selected action is an execution step s , the same-debt target is $\text{row}(x_{k+1})$, and the budget drops strictly (Lemma S1.1); composed with empty histories on both sides it is a payment macro of Definition 6.6. The target budget is at most the current budget, so there is no refill; the continuation relation strictly decreases the budget and is well founded.
3. *Settlement.* At $k = B$ we have $r(x_B) = 0$, so the expression of x_B is a constant $\text{const}(v)$ and a local settlement exists; a current of budget zero has no payment macro. The paid trace $t_B : e \rightarrow \text{const}(v)$ has length B (Lemma S1.2), and soundness gives $v = \text{ev}_\rho(e)$. By Lemma S1.1, $x_{B+1} = x_B$; the visit count keeps advancing, and visits of different causal depth are still distinguished in finite histories.
4. *Receipt.* By S0.3 the relation image of t_B is $e - \text{const}(\text{ev}_\rho e) \in \ker(\text{ev}_\rho)$; for every increment ε , its value under $\rho + \varepsilon$ is $\text{eff}_{\rho, \varepsilon}(e)$. ■

Direct use. Proposition 6.11 applies this theorem with the whole-tree program as reader; Theorem 7.7 and every run of §9 use the same owner-free installation as their mathematical row, reading its value, charged history, original material and relation boundary.

Formal correspondence. JS/OwnerFree/Source.lean (vocabulary, eventAlgebra, whole, mathEntry, destination_math); OwnerFree/Compiler.lean (remainder, patch, patch_fold, compiler, ledgerRoot); OwnerFree/Completion.lean (next_budget, budget, history_accounting, completed_value, completed_history_length, completed_next_state); OwnerFree/Payment.lean (lin-

eage, debtCurrent, activePayment, wellFounded, no_refill, endpoint_no_paid); OwnerFree/Consumer.lean (value_source, paid_history, relation_boundary, updated_inverse_fibre). Commit AB.

S2 Theorem 8.12 and Theorem 7.7

S2.1 Theorem 8.12: the actual action generates the next question

Objects. An inquiry frame Φ : an inquiry state, the complete native program reading the root compiler, the raw input (ρ, e) registered at the current visit, a reader E giving the actual environment at every original occurrence, and a causal depth k . The mathematical state of the frame is a paid state (e', t) of $\Lambda_{\rho, e}$; ρ' is the reading of E at the frame's occurrence and $\varepsilon = \rho' - \rho$.

(1) Value of the request. The residual request $e \ominus e' = e + \text{linear}_{-\text{id}}(e')$ is registered on $\rho + \varepsilon = \rho'$. By the definition of evaluation $\text{ev}_{\rho'}(e \ominus e') = \text{ev}_{\rho'}(e) - \text{ev}_{\rho'}(e')$. Soundness gives $\text{ev}_{\rho}(e) = \text{ev}_{\rho}(e')$, so the old value is zero. The relation image of the paid trace is $e - e'$ (S0.3), whose value under ρ' is by (S0.2) $\text{eff}_{\rho, \varepsilon}(e) - \text{eff}_{\rho, \varepsilon}(e')$, in agreement with the request's value. By S0.4 the canonical residual of this certificate under $\text{ev}_{\rho'}$ vanishes iff its value vanishes, i.e., iff the request's value vanishes. At a settlement $e' = \text{const}(\text{ev}_{\rho}e)$ and the request's value is $\text{eff}_{\rho, \varepsilon}(e)$.

(2) Necessary birth. $r(e \ominus e') = r(e) + (r(e') + 1) + 1$: the addition node contributes 1 and the linear node 1. This is positive, so the initial state is not a constant and the executor's first action is an execution step; in the birth compilation this step compiles to an obligation admission (the birth of a new responsibility), not to a payment of any old debt row.

(3) Recursive frames and run. Define the **rank** of a frame $\text{rk}(\Phi) := (r(e), k + 1)$, ordered lexicographically. The next frame is determined by the current action:

- an execution step gives the same frame with depth increased by one, of rank $(r(e), k + 2)$;
- a settlement gives the born frame—old state the current inquiry state, program the successor program, registered input the residual request, depth zero—of rank $(r(e) + r(e') + 2, 1)$.

Both cases strictly increase the rank. The sequence of frames Φ_n (Φ_{n+1} the next frame of Φ_n) is therefore strictly increasing lexicographically. The erasure of a frame determines its rank: the raw input is read back by the root's observation (erasure keeps vocabulary and visit, and the observation reads back the registered raw input), and the causal depth is read from the history of the erased visit. If Φ_m and Φ_n had the same erasure with $m < n$, they would have the same rank, contradicting strict increase. So erasure is injective. Validity of successors is checked branch by branch: an execution step uses the successor law of a direct answer, a settlement uses the successor law of an obligation admission, and both preserve the generated living law on the same root. The node of the existing canonical inquiry runtime at count n is the presentation of Φ_n : by induction on n , the activation successor of a node is the next frame. Every frame has a singleton query fibre (its only query is the frame's registered input), so no external table of future requests is needed.

(4) Separation of payment and birth. Restrict actual nodes to the debt process of the fixed root: the execution-step branch emits a same-debt payment receipt on the same mathematical row, whose whole-ledger destination equals the ledger evolution of the joint execution step (the reading of Theorem 6.10(2)); there is no payment macro where the budget is zero (Theorem 6.10(3)); the birth of the settlement branch is a new obligation admission. ■

Formal correspondence. `JS/Native/ResidualRequest.lean` (`expression_eval`, `old_value`, `budget`, `action_is_paid`, `updated_value`, `residual_zero_iff`); in `JS/Native/Restructuring/Inquiry/Continuation/`: `Policy.lean` (`rank`, `Before`, `next_progress`, `frames_forward`), `Inventory.lean` (`rank_eq_of_erasure`, `frames_erase_injective`), `Occurrence.lean` (`successor_valid`), `Runtime.lean` (`actual_node`, `macro_next`, `macro_next_preserves`), `Payment.lean` (`actual_paid_receipt`, `no_paid_of_zero`). Commit AB.

S2.2 Theorem 7.7: shared run of the registered input and successor feed

Objects. The fixed data of Proposition 7.6: an old inquiry state (`root`, `visit`, `audit protocol`, `calculus`, `query type`, `bearers` and `authority`), a recognition installed on a complete complex inner-product space $\mathcal{H}$, the step at this visit and its disposition `settle`; in addition, the original inquiry input ι registered by the root at this visit: a projection position, its activation proof at this occurrence, a classifier equation and a query-type equation.

(1) Registered activation. Split by the output of `settle`.

All-generated or joint-residual branch. Proposition 7.6 gives a program package and the calculation inquiry state, whose root is the root with the calculation inquiry installed. On this root add a **query projection law**: the projection type is a singleton; at an occurrence o' it is active iff o' is the exact occurrence of this visit; the payload type is the calculation query; the projected value is the calculation query whose incidence is the original query of ι . The new root r_ι is the original root with this projection added; the visit is unchanged. The registered input ι' takes this projection, the activation proof at the exact occurrence and the classifier equation (at the exact occurrence the classification takes the active branch by definition); the query-type equation holds by definition. The inquiry state s_ι has root r_ι , keeps `visit`, `audit protocol` and `calculus` of the old state; `bearers` are read through the query's incidence; `authority` is the old `authority` lifted along the new projection face; the compilation program answers every query with the result face of the package embedded through the projection.

So s_ι compiles the query of ι' to an answer (by definition). The expression read by the answer face is the whole-tree program of the package, so the parser reads T back (Theorem 3.9(3) and the tree equation of Proposition 7.6); the value is $\delta_{\tau(T)}$ (Theorem 3.9(1)); the source state is the paid state of the package. The compilation face of the old state for every original query q is read back through the same embedding and still reads the canonical token of the old compilation.

The other four branches. The terminal branch returns the whole-ledger compilation of the root compiler at the source occurrence, and the three residual branches return the residual object itself, as in Proposition 7.6.

(2) Shared run. With s_ι as inquiry state and the constant function “return the raw input of the query of ι' ” as reader, construct the mathematical inquiry runtime (the owner-free installation of Theorem 6.10 plus the runtime of §8.5). The raw input of the owner-free installation is the raw input of this query; by Theorem 3.9(3) its expression reads back T . Theorem 6.10(3) gives a charged history of length $r(\text{prog}_\kappa T)$, rewritten to $\beta(T)$ by Theorem 3.9(2), and the completed value $\text{ev}(\text{prog}_\kappa T) = \delta_{\tau(T)}$. The whole-ledger write-back and original material of the mathematical

row at every count, and the actual query, answer and successor at every runtime offset, are the node equations of the mathematical inquiry runtime at that count and offset (Theorem 8.12(3)).

(3) Successor feed. The inputs are only the living root, the visit v , the recognition and the audit protocol. On nonterminal branches the step carries a successor succ and the equation $\text{next}(v) = \text{succ.current}$; put $v^+ := v.\text{next}(\text{succ})$. Recognition generates a step at v^+ and settle classifies it; the constructions of (1)(2) give again a feed and a run at v^+ . The terminal branch returns only the whole-ledger compilation. Every nonterminal branch returns the preceding program package (the two aligned branches) or residual (the three residual branches) paired with the next run.

Whole-ledger write-backs and next currents of the two steps: the definition of a step contains $\text{whole}(\text{step}(v)) = \text{generatedLedgerAt}(v.\text{current})$ and $\text{nextCurrent}(\text{step}(v)) = \text{generatedNextCurrentAt}(v)$, and the same holds at v^+ . The successor's target occurrence equals the root's canonical occurrence at the current of v^+ (the target occurrence is the event the compiler emits for the next current), and history is a function of the occurrence, so the two histories are equal. ■

Direct use. The joint program of §9 (Theorem 9.2) takes the same recognition and the same successor as input; the default run of Theorem 9.8 consumes the next run of (3) at every visit.

Formal correspondence. `DJ/Input/{Source,Consumer}.lean, DJ/Input/Runtime/{Source,Consumer}.lean, DJ/Next/{Source,Consumer}.lean`; low source inquiry `JS/OwnerFree/Installation/Math/{Inquiry,Frame}/Source.lean`. Commit AC.

S3 Theorems 9.2 and 9.3

S3.1 Theorem 9.2: one program carries the complete time, the common history and the action difference

Objects. A living root, a temporal visit v , a recognition and an audit protocol; $\text{step}(v)$ lies in the all-generated or joint-residual branch, with successor v^+ , history transition and pairing alignment; T is the aligned constructor tree of Proposition 7.6.

(1) Temporal codes. A temporal visit of the root is given by one of three inductive histories: finitely reachable from the initial current, sealed cofinal, or reachable after cofinality. For each, encoding and decoding between the inductive types `Hist`, `Post` and the original reachability evidence are mutually inverse, defined and verified constructor by constructor. A temporal code is $c = (\text{current}, \text{history code})$; `enc` and `dec` take these encodings and decodings componentwise, so they are mutually inverse and `enc` is injective. The first component of $\text{act}(c) = (\text{rcpt}(c), \text{nx}(c))$ contains c itself, so `act` is injective. Successor: the successor of a temporal visit is given by the compiler's next current together with the extension of the history, and the encoding commutes with the extension constructor by constructor, so $\text{enc}(v^+) = \text{nx}(\text{enc } v)$.

(2) Joint program. Nodes are pairs (aligned node of T , temporal code), and T_J gives every node of T the code $\text{enc}(v)$. Results are five-tuples

(effect fold, receipt tree, original material of both histories,
evaluator inputs of both sides, exposure plans of both sides).

The constructor $\kappa_J(n, \vec{y})$: its first component is the single-node fold of the original dependent effect $\text{effectFold}(n_1, [y_{i,1}])$; its second component is the tree with root $\text{act}(n_2)$ and the receipt trees of the children as subtrees; the last three components are the original material of both histories, the evaluator inputs of both sides and the exposures of both sides, independent of the children. The program is $P = \text{prog}_{\kappa_J} T_J$ (Definition 3.8). The environment δ sends every node variable to its state point; δ' sends every node n to δ_{n^+} with $n^+ := (n_1, \text{nx}(n_2))$ (keeping n_2 when there is no successor).

Value. Mutual recursion on T_J and child lists gives

$$\text{ev}_{\delta'}(P) = \delta_{\text{fold}_{\kappa_J}(T_J^+)},$$

where T_J^+ is the tree with every node replaced by n^+ —the same as the proof of Theorem 3.9(1), with base points n^+ . By S0.2 the value of $\tilde{P}$ in the pair environment $(\delta, \delta' - \delta)$ is $(\text{ev}_{\delta} P, \text{eff}_{\delta, \delta' - \delta} P)$, and the update equation (Proposition 3.3) gives $\text{eff}_{\delta, \delta' - \delta}(P) = \text{ev}_{\delta'}(P) - \text{ev}_{\delta}(P)$. Together the value is $(\delta_F, \delta_{F'} - \delta_F)$ with $F = \text{fold}_{\kappa_J} T_J$ and $F' = \text{fold}_{\kappa_J} T_J^+$.

Components. By induction on T_J : the first component of F depends only on the first components of the nodes and of the children, and is exactly $\text{fold}_{\text{effectFold}}(T) = \tau(T)$; the second component of F is the image tree of T_J under act applied node by node, with root $\text{act}(\text{enc } v)$, which decodes back to v by (1).

Charge and payment. By S0.2 and Theorem 3.9(2), $r(\tilde{P}) = r(P) = \beta(T_J)$, and T_J has the shape of T , so $\beta(T_J) = \beta(T)$. Applying Theorem 6.10 to the reader $o \mapsto (\text{pair environment}, \tilde{P})$: the paid trace has length $\beta(T)$, every count $k < \beta(T)$ has a strict payment, the target budget is never refilled and the continuation is well founded.

Inverse reading and successor. At every count the expression kept by the installed face is P , and the parser reads back T_J (Theorem 3.9(3)); forgetting the code component reads back T . The recovered root code decodes to v , and its generated next current is the root's generated next current at v . The update morphism $(\text{id}, \text{ev}_{\delta'})$ goes from ev_{δ} to $\text{ev}_{\delta'}$; on the source side take the relation words of the old trace, and as target fibre element the sum of the relation words of the old and new traces. By Lemma S0.5 the lifting residual vanishes iff the target fibre element is the image of an element of the old fibre; this residual is kept unchanged with the result.

(3) Common history. The source and target histories are histories of the same history law at two occurrences; events, supports, generator closures and relation closures of the common history are the unions of both sides. The two transitions embed the generators of each side by definition. The completion map sends a word u of the generator closure of one side to the class of the same word in the common completion, so its image vanishes iff u lies in the relation closure of the common history. The evaluators of the two sides are the faithful evaluations of the history law at the two occurrences, reading back the values of the original faithful faces on every generator; relation

soundness is equivalent on both sides. The product of the completion carriers Src and Tgt carries the action and observation induced by the two transitions and forms an action model; pairing and dual action come from the scalar pairing of the histories; the measurement $\text{Src} \times \text{Tgt} \rightarrow \ell^2(\mathcal{H} \times \mathcal{H})$ together with a linear isometric evolution of that space forms joint integral-coherent action data.

(4) Inverse criterion of the action difference. Write ∂_s, ∂_t for the differentials (measurements) of the two sides; the transition satisfies $\partial_t \circ \iota = \partial_s$.

If there is r with $\partial_s r = \partial_s a_s$ and $\iota(r) = a_t$, then $d = \iota(r) - \iota(a_s) = \iota(r - a_s) \in \text{range } \iota$, so the image residual vanishes, and $\partial_t d = \partial_s(r - a_s) = 0$, so the observation residual vanishes.

Conversely, a vanishing image residual gives r_0 with $\iota(r_0) = a_t$; then $d = \iota(r_0 - a_s)$ and the observation residual $\partial_t d = \partial_s(r_0 - a_s) = 0$, so $r_0 \in \text{Fib}_{\partial_s}(a_s)$.

The computing program is a three-output expression: the difference d , its measurement, and its class in the cokernel; its remaining count is 25. Theorem 6.10 gives a paid trace of 25 steps, each a strict payment, with well-founded continuation, and whole ledger and next current kept at every count.

(5) Literal successor. The target occurrence of $\text{step}(v)$ is the event the compiler emits for the next current, equal to the canonical occurrence at the current of v^+ , which is the source occurrence of $\text{step}(v^+)$. Histories, generator closures, relation closures, exposures and exposure trees are functions of (current, occurrence), hence pairwise equal; target words are carried to next source words along the equality of generator closures, with the underlying word unchanged. ■

Formal correspondence. Occurrence/Temporal/{Source,Action/Source,Action/Inverse}.lean; Occurrence/Temporal/History/Common/{RootSource,Closure,Evaluator/Source,Action/Source}.lean; DJ/Joint/{Source,Laws,Consumer,TargetDifference,TargetProgramme,TargetConsumer}.lean; DJ/Next/{Material,Pursuit}.lean. Commit AD.

S3.2 Theorem 9.3: joint recovery by all actors

Objects. The data of Theorem 9.2 and a count. The source and target pairing unfolding trees have all pairing positions as nodes, each with an exposure (M_p, A_p, U_p) . Actors are members of the tree traces; the node Hilbert space is indexed by the positions of the traces (the same payload occurring repeatedly is counted at each position).

(1) All-word model. The joint carrier is the product of the carriers of the two sides; the action of a letter ℓ is the carrier action of that actor (source actors act on the first factor, target actors on the second, and “simultaneous” acts by the roots of both sides), and the actions of the root actors are the source and target exposure actions of the original step. The read-out rd is the product of the coarse reader and the measurements of all actors. Lemma S0.6 gives the quotient Mod , the fibre characterization and the completion equivalence; two elements of the completion carrier are equal iff their read-outs after all following words agree, which is S0.6(ii) stated on the completion side.

(2) Ordered effect history. Take $W = \mathcal{H}^{\text{actors}}$, let U_ℓ apply U_p to the component of the selected actor and leave the others unchanged (“simultaneous” applies to the root components of both sides), and let M be the measurements of the actors. Lemma S0.7 gives that the total residual

is the sum of the step residuals transported by the later evolutions, with history length $|w|$; for a single letter the total residual is that actor's covariance effect $U_p M_p(x) - M_p(A_p x)$.

(3) Node Hilbert. The measurement $\text{meas} : \text{Mod} \rightarrow \ell^2(\text{positions}; \mathcal{H})$ takes at every position the feature of that actor (the actor's measurement read through the model). In ℓ^2 the squared norm is the sum of the squared norms of the coordinates, so

$$\|\text{meas}(x)\|^2 = \sum_p \|M_p(x)\|^2.$$

The evolution of a letter on ℓ^2 takes U_p or the identity coordinatewise and is therefore a linear isometry. The node effect unfolds coordinatewise by the definition of the covariance coupling residual.

(4) Future recovery and three faces.

Future recovery. Let $x \in \text{Mod}$ be sent to zero by both the coarse reader and all future readers. The vanishing of all future $\mathcal{H}$ readers means: for every prefix bound and every word, every actor's measurement after the word agrees with that at the zero point. The vanishing coarse reader gives agreement of the coarse read-outs after every word. So x and 0 have the same read-outs after every word, and (1) gives $x = 0$. The combined map is injective, Mod is linearly isomorphic to its image, and both composites are identities.

Three faces. With $I = \text{Mod}$ (acted on by the model advance), $H =$ the family of ℓ^2 spaces over all prefix bounds (acted on by the evolution at each bound), $Q =$ the coarse model (acted on by the coarse advance), c the future reader and m the coarse restriction, form the complete orbit carrier $\mathcal{C} \subseteq I \times H \times Q$ of Lemma S0.8. The integral and coarse faces are the first and third components. The coherent face sends the second component (the family over all prefix bounds) to the inverse limit along restriction of prefix bounds: restriction at each bound commutes with evolution, so this is the completion map of a compatible family and commutes with the evolution of every letter; each bound coordinate of the completion map is the value of the second component at that bound, so the coherent face vanishes iff the second component vanishes. An element on which all three faces vanish therefore has all three components zero, hence is zero: the three faces are jointly injective, give a double inverse on the image and commute with the action of words (S0.8(ii)).

(5) Paid recovery. For a point, a seed word, a history word and a query word q , the owner-free calculation executes the program generated by the query word, whose remaining count is $|q| + 3$ by the program-budget lemma; Theorem 6.10 gives a paid trace of length $|q| + 3$, and the computed value equals the next value (value-readback lemma). Applying the recovery map to the acted element reads back the next value as first component through the isomorphism of (4). The pair inventory enters the next birth as part of the born inventory, and the same debt is never refilled (Theorem 6.10(2)). ■

Formal correspondence. `AW/{Source,Complete/Source,EffectHistory/Equation,EffectHistory/Source}.lean; AW/NodeHilbert/{Source,Future/Recovery/Source}.lean; AW/NodeHilbert/Future/Wave/{Words/Kernel,Full/Source,Recovery/Source,Recovery/Consumer,Incidence/Consumer}.lean. Commit AD.`

S4 Theorems 9.5–9.7

S4.1 Theorem 9.5: mother admission

Objects. An inquiry frame Φ and a source program $\mathcal{S} = (L, \text{reader}, \dots)$ (Definition 9.4). The shared runtime installs Φ and the reader of $\mathcal{S}$ as an inquiry state and answers its unique query; s is this answered state, o its current occurrence, and ρ' the environment of the raw input of the query of the shared runtime’s literal next state.

(1) Material and value. The material is assembled by definition: environment ρ and expression e are the output of the reader at o , the increment is $\varepsilon = \rho' - \rho$, the paid state (e', t) is the result of the shared runtime at o , and the bearer is the bearer of this query at s . The registered input is the residual request of Definition 8.11, with environment $\rho + (\rho' - \rho) = \rho'$. Value and residual are given by S2.1(1).

(2) Necessary admission. If the initial state of the residual request were a settlement, its budget would be zero (the execution debt law has budget zero at settlements); but the residual request has budget $r(e) + r(e') + 2 > 0$ (S2.1(2)), a contradiction. So the source’s first action is an execution step. The birth program is generated by the old state, the query, the registered input, the source packet, the authority and this step; on the root and visit of s , the compilation program sends the unique query to what the birth program generates on the exact temporal causal event—an obligation admission—and the equation holds by definition.

(3) Target. The admission target carries: a literal next current equal to (vocabulary, root, visit) of the target state; the first whole-ledger write; and, for every old projection, an outcome on the target equal (heterogeneously) to the original outcome. Validity of the successor is given by the general criterion “an active node preserves the generated living law under an admission successor”, whose hypotheses are exactly the literal next current and the compilation equation above.

(4) Born frame. The born frame takes s as old state, the residual request as registered input, the reader environment, depth zero, and as inventory the complete exposure of the paid trace. Its current state is by definition the admission target state. The runtime of Theorem 8.12 starts from it: the erasure of the initial node equals the erasure of the target presentation; the macro successor and preservation of the living law at every tick are the node equations of S2.1(3); no refill and well-foundedness are S2.1(4) and Theorem 6.10(2).

(5) Next content. By (S0.1) and the update equation, the sum of (old, effect) of the paid result equals the program’s value in the updated environment:

$$\text{ev}_\rho(e) + \text{eff}_{\rho, \varepsilon}(e) = \text{ev}_{\rho + \varepsilon}(e).$$

The environment of the born frame at every occurrence is given by the environment read-out of this value. The macro-history program satisfies $\text{next}(\text{hist}_k) = \text{hist}_{k+1}$; the active environment of the initial frame is already the next value of the history of tick k , i.e., tick $k + 1$, and the born frame advances one more tick, giving tick $k + 2$. The born inventory is the side-by-side seed of the old inventory and the paid exposure, both of which are parts of it (left and right embeddings).

(6) Native shift. The native cursor shift of the query frame of the same stage gives a new frame; applying (1)–(4) to it gives compilation, first whole-ledger write, validity of the successor,

literal successor, the run at every offset, no refill and well-foundedness; the shifted environment is the actual environment of the cursor shift, and effect and inverse reading are read from the residual request. ■

Formal correspondence. ACT/Programme.lean, ACT/Receipt/{Source,Execution,Compiler,Consumer,Runtime}.lean, ACT/Target.lean, ACT/Receipt/Faces/{Source,Action,Consumer,Native/Source,Native/Consumer}.lean; next content AW/NodeHilbert/Future/Wave/Incidence/Environment/{Source,Consumer}.lean. Commit AD.

S4.2 Theorem 9.6: typed inventory crosses the admission

Objects. The setting of Theorem 9.5; the physical raw input (ρ_P, e_P) (variables of physical type Var_P) with its paid result, and the low-level raw input (ρ_L, e_L) (variables in L) with its result. The binding is $\sigma : \text{Var}_P \rightarrow \text{Expr}_L$, $\sigma(x) = \text{const}(\rho_P(x))$.

(1) Binding preserves values. Structural induction proves $\text{ev}_{\rho_L}(e[\sigma]) = \text{ev}_{\rho_P}(e)$: at a variable $\text{ev}_{\rho_L}(\text{const } \rho_P(x)) = \rho_P(x)$; at a constant both sides agree; addition, linear and bilinear nodes agree term by term by the induction hypothesis. Formal words extend linearly. Hence the value of the low-level query $(\rho_L, e_P[\sigma])$ equals that of the physical query. The substituted trace is the physical paid trace substituted along σ , starting at $e_P[\sigma]$, with its relation boundary substituted accordingly; the physical paid state is a constant and remains a constant after substitution. Every execution trace ending at a constant has length equal to the remaining count of its start (Proposition 3.7(1)), so the substituted trace has length $r(e_P[\sigma])$.

(2) Registration and readback of the inventory. The complete material (both raw inputs and results, the binding, both written inventories and the substituted trace) is installed as one projection on the base root of the frame, with payload type the type of the material. The projection law of the mother admission target inherits the old projection law (Theorem 9.5(3)), and its outcome at this projection is heterogeneously equal to the original outcome; the equality of types is given by the same heterogeneous equality, and transporting the outcome along it to the material type gives the material itself.

(3) Born inventory. Every event of the physical inventory is carried into the low-level language by substitution along σ and placed beside the low-level inventory to form the born inventory; the left and right embeddings ensure that every event of both sides is in it. The born frame runs as in Theorem 9.5; the next inventory extends the born inventory, and the coordinate map is injective under both actions mathNext and nextBorn and reads back every original event.

(4) Inverse readback. Index a word family W by the events of the born inventory, with joint map $W \mapsto (\text{index} \mapsto (\text{ev}_{\rho}, \text{eff}_{\rho, \varepsilon})(W_{\text{index}}))$; the canonical residual of the actual word family vanishes iff its image vanishes (S0.4), and that image is the result value. The next inverse readback is the map induced by the update morphism (S0.4). At every tick of the cofinal stage, the pair value of a relation word in that tick's environment reads the value of the next tick's field at offset zero; the paid trace has length equal to the remaining count of the raw input. ■

Formal correspondence. ACT/Receipt/Inventory/{Source,Laws,Installation}.lean; Inventory/Born/{Source,Consumer}.lean; Inventory/Born/Vector/Fibre/{Source,Consumer}.lean; Inven

tory/Born/Coordinates/Actual/{Source,Query}.lean; Coordinates/Actual/Cofinal/{Source,Consumer}.lean. Commit AE.

S4.3 Theorem 9.7: word–state pairing and paid prefixes

Objects. A runtime and its source; the old environment ρ_s and increment $\varepsilon_s = \rho_{\sigma(s)} - \rho_s$ of a state s . The binding σ of orbit variables satisfies two readbacks: $\text{ev}_{\rho_s}(\sigma x) = \rho_{\sigma(s)}(x)$ and $\text{eff}_{\rho_s, \varepsilon_s}(\sigma x) = \varepsilon_{\sigma(s)}(x)$.

(1) Pairing. On base points δ_s , $\langle w, \cdot \rangle$ takes the stage inventory $(\text{ev}_{\rho_s} w, \text{eff}_{\rho_s, \varepsilon_s} w)$, extended uniquely and linearly by the universal property of free modules (Proposition 2.3); linearity in w comes from linearity of evaluation and effect.

(2) Covariance and coimage action. On base points,

$$\langle w[\sigma], \delta_s \rangle = (\text{ev}_{\rho_s}(w[\sigma]), \text{eff}_{\rho_s, \varepsilon_s}(w[\sigma])) = (\text{ev}_{\rho_{\sigma(s)}} w, \text{eff}_{\rho_{\sigma(s)}, \varepsilon_{\sigma(s)}} w) = \langle w, A\delta_s \rangle,$$

where the middle step is the substitution lemma—evaluation after substitution equals evaluation in the environment “variables take the values of their substitutes”, and likewise for effects—and the two readbacks identify that environment with $\rho_{\sigma(s)}$ and $\varepsilon_{\sigma(s)}$. Both sides are linear on $\mathbb{Z}[S]$, so $\langle \sigma^*(\cdot), \cdot \rangle = \langle \cdot, A(\cdot) \rangle$ on the whole carrier. This says (σ^*, A^*) is a morphism from the pairing to itself, which induces the action on the coimage by S0.4; the read-out formula on $\text{can}[e]$ holds by definition.

(3) Iteration is time. Induction on n : $n = 0$ is trivial; the step uses the base-point equation of (2) at tick d and the induction hypothesis at $d + 1$. $\text{act}^n[e] = [e[\sigma]^n]$ follows from commutation of the induced map with the canonical map.

(4) Prefix charges. The i -th prefix executes $\widetilde{e[\sigma]^i}$, and by S0.1 and S0.2 its trace has length $r(e[\sigma]^i)$; summing gives Σ_n . The batch query places the $n + 1$ terms in a vector-valued language: each term enters through one coordinate-embedding node and one accumulation node, so the remaining count is the sum of the terms plus $2(n + 1)$; execution length equals remaining count (Proposition 3.7(2)). Theorem 6.10 gives strict payment at every step and same-debt lineage.

(5) Call recovery. The actual query material of a call frame at count $c + i$ gives the environment there; the old values and increments of the snapshots needed are taken entry by entry from these environments (an increment is the difference of two adjacent ones). The batch environment so assembled equals the orbit batch environment coordinate by coordinate by definition, so the result values of the two queries are equal; the paid trace of every call has length equal to its remaining count (Theorem 6.10(3)), and summing gives call charge equal to call cost.

■

Formal correspondence. Operations/Inquiry/Context/Native/Pairing/{Source,Orbit/Source,Orbit/Action,Orbit/Iteration,Orbit/Iteration/Batch,Orbit/Iteration/Current}.lean; calls and history $\cdots/\text{Batch/Calls}/{\text{Source,Consumer,History/Source,History/Consumer}}.lean$ (full prefix in Appendix D.2 of the main text). Commit AE.

S5 Theorem 9.8: the default run executes the source-generated request and its subtrees

(1) Source-generated request. Take the root and installed reader of the joint program of Theorem 9.2, form the source inquiry frame of the owner-free installation, and take as seed the zero unfolding of the generator of the registered expression. The source program of the default runtime has “the original request of this occurrence” as reader: at an occurrence o the reader returns the request expression and environment generated for o . Apply Theorem 9.5 to this source program: the raw input of the query is the request read by the installed face (by definition); the result value equals the execution value of the request (the completed value of Theorem 6.10 together with soundness of ev); the paid trace has length equal to the remaining count of the registered input (Theorem 6.10(3)); by Theorem 9.5(5) the born pair inventory consists of the previously written part and the part paid now; the actual query, answer and successor at every count are node equations of the runtime. The ledger roots agree by construction. In the branch version the core and relation expressions are the two coordinates of a binary batch, and the trace and substituted trace of the batch execution enter the initial inventory.

(2) Subtrees enter the main expression. The main expression is a sum of three terms: the original query expression, the subtree at the old code and the subtree at the next code, each entering the output type through a coordinate embedding. The subtree at a code is the zero constant if recognition has no successor at the visit of that code, and otherwise the embedding of the raw input of the subprogram at that code. Hence

$$ev(\text{main}) = (ev(\text{query}), \text{ch}(c_{\text{old}}) + \text{ch}(c_{\text{next}})),$$

where the first component of a subtree value is zero. Remaining counts add term by term, plus two additions and one linear embedding, three combining nodes in all: $r(\text{main}) = r(\text{query}) + c_{\text{old}} + c_{\text{next}} + 3$. Paid trace length: the trace of the owner-free installation has length equal to the remaining count, and the query’s own paid trace has length $r(\text{query})$; substituting gives the claim. The executions of the two subtrees have length equal to their own remaining counts (Proposition 3.7(2)). Events of the old inventory and of the new payment are kept by the embeddings of the side-by-side seed; the sum of the old and next actor components is the state point of this tick’s source operation, read back through the complete actor carrier.

(3) Subtree environments. At a supplied occurrence the subtree environment is defined by the read-out of the original binding in that environment; the environment of the born frame is the actual updated environment (Theorem 9.5(5)), and the subtree environments are read from it by the same binding. Scalar and pair inventories are kept as in (2); the initial node and the input at every tick of the runtime are the runtime’s defining equations. ■

Formal correspondence. ACT/Inventory/Pair/Residual/Joint/Past/NextObservation/Request/Acted/{Source,Consumer}.lean; DJ/{Joint/Branch}/Primary/{Source,Consumer}.lean; DJ/Branch/JointQuery/Actor/{Source,Embedding,Action/Source,Operation/Source,Operation/Consumer}.lean and Actor/Operation/Query/Inventory/{Language,Source,Consumer,Action/Source,Action/Consumer,Live/Source,Live/Consumer}.lean; default successor DJ/Next/{Source,Consumer}.lean.

Commit AE.

S6 First-release selection numbers and proof locations

The table matches claim numbers of the first-release selection to results of the main text and to the written proofs. The selection numbers agree with `core.Cn` in the first-release map of the public code selection (`H0mework docs/first-release-map.json`), so that each item can be checked.

Selection number	Result in the main text	Written proof	Source commit
core.C1	§6.1 Proposition 6.1	main text	H (base regression repair)
core.C2	§6.4 Definition 6.6, Theorem 6.7, Remark 6.8	main text	H
core.C3	§2.1 Definition 2.1, Proposition 2.2	main text	H
core.C4	§8.4 run period	main text	H
core.C5	§8.4 Proposition 8.9, Table 4, Figure 5	main text; Appendix C.3	H
core.C6	§8.4 Proposition 8.10	main text	H
core.C7	§3.1–§3.3 Definitions 3.1–3.2, Propositions 3.3–3.6	main text	H
core.C8	§4 Theorems 4.4, 4.6, 4.7, Proposition 4.8	main text; Appendices A–B	H
core.C9	§2.2–§2.4 Proposition 2.3, Definition 2.4, Proposition 2.5	main text; Appendix A.1	H
core.C10	§5 Definition 5.1, Proposition 5.2, Theorem 5.3	main text	H
core.C11	§7.1–§7.3 Proposition 7.1, Definition 7.2, Propositions 7.3–7.5	main text; Appendix C.6	H
core.C12	§6.2–§6.3 Definitions 6.2–6.3, Propositions 6.4–6.5, Table 2	main text; Appendix C.5	H
core.C13	§8.1–§8.3 Definition 8.1, Propositions 8.2–8.3, Theorem 8.4, Definition 8.5, Theorems 8.6–8.7, Proposition 8.8	main text; Appendices C.1–C.4	H
core.C22	§6.5 Definition 6.9, Theorem 6.10	S1	AB
core.C23	§3.4 Proposition 3.7, Definition 3.8, Theorem 3.9; §6.5 Proposition 6.11; §10.1	main text (complete)	AB
core.C24	§8.5 Definition 8.11, Theorem 8.12, Example 8.13, Figure 6	S2.1	AB
core.C25	§7.3 Proposition 7.6	main text (complete)	AB

Selection number	Result in the main text	Written proof	Source commit
core.C26	§7.3 Theorem 7.7(1)(2)	S2.2	AC
core.C27	§7.3 Theorem 7.7(3)	S2.2	AC
core.C28	§9.1 Definition 9.1, Theorem 9.2	S3.1	AD
core.C29	§9.2 Theorem 9.3	S3.2	AD
core.C30	§9.3 Definition 9.4, Theorem 9.5	S4.1	AD
core.C31	§9.3 Theorem 9.6	S4.2	AE
core.C32	§9.3 Theorem 9.7	S4.3	AE
core.C33	§9.4 Theorem 9.8(1)	S5	AE
core.C34	§9.4 Theorem 9.8(2)(3)	S5	AE
